Napredno strojno učenje: Uvod v predmet in meta učenje

Ljupčo Todorovski

Univerza v Ljubljani, Fakulteta za matematiko in fiziko
Institut Jožef Stefan, Odsek za tehnologije znanja (E8)

Oktober 2026

Pregled predavanja

1. Kaj moramo vedeti od prej
2. Pravila igre
   - Vsebine in ocenjevanje
   - Kratka predstavitev vsebinskih sklopov
3. Neobstoj brezplačnega kosila (NBK)
   - NBK za Boolove funkcije
   - Posledice izreka NBK
4. Meta učenje
   - Osnovni okvir
   - Meta cilji in povezava z AutoML

Kaj moramo vedeti od prej

Spremenljivke in njihovi tipi

Spremenljivka X z domeno D_X

Numerične vrednosti izražajo količine. Spremenljivka je lahko diskretna (število nakupov) ali zvezna (višina).

Kategorične vrednosti označujejo kategorije. Te so lahko nominalne (neurejene) ali ordinalne (urejene).

Dvojiška (binarna) spremenljivka

Ima dve možni vrednosti: $|D_X| = 2$. Običajni domeni sta $\{0, 1\}$ in $\{-1, 1\}$.

Podatkovna množica je seznam primerov

Nadzorovano učenje

$$S = (e_1, e_2, \ldots, e_n), \qquad e_i = (\boldsymbol{x}_i, y_i), \qquad \boldsymbol{x}_i = (x_{i1}, x_{i2}, \ldots, x_{ip}).$$

- n: število primerov; $i = 1, \ldots, n$: indeks vrstice.
- p: število napovednih spremenljivk; $j = 1, \ldots, p$: indeks stolpca.
- $x_{ij} \in D_{X_j}$ in $y_i \in D_Y$: opazovani vrednosti.

Izraz podatkovna množica ohranimo, vendar je S indeksiran seznam. Enaki primeri se lahko ponovijo; vsak pojav šteje posebej.

Notacijski dogovori

Spremenljivke in vrednosti

- $\boldsymbol{X} = (X_1, X_2 \ldots, X_p)$ je seznam napovednih spremenljivk
- Y je ciljna spremenljivka
- $\boldsymbol{x}$ je splošen vektor vhodov in $\boldsymbol{x}_i$ je vektor vhodov za i-ti primer
- $\hat{y}_i = m(\boldsymbol{x}_i)$ je napoved modela in y_i je opazovani cilj

Od seznama do tabele

- $X = (x_{ij})$: je tabela vhodov z n vrsticami in p stolpci
- $\boldsymbol{y} = (y_1, y_2, \ldots, y_n)$ je seznam ciljnih vrednosti

Pri numeričnih vhodih je X matrika; kategorične vrednosti pred uporabo številskih metod ustrezno kodiramo.

Podatkovna množica za nenadzorovano učenje

Seznam brez podane ciljne spremenljivke

$$S = (\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_n), \qquad \boldsymbol{x}_i = (x_{i1}, x_{i2}, \ldots, x_{ip})$$

Tudi tu se lahko enaki primeri ponavljajo.

Iščemo podatkovne vzorce

- Skupine primerov s podobnimi lastnostmi
- Vrednosti spremenljivk, ki se pogosto pojavljajo skupaj

Ponazoritev: kartica zvestobe

Izmišljeni podatki: vhodi ob začetku leta, cilj v naslednjih 12 mesecih.

Ime	Prihodki EUR/mesec	Starost leta	Spol	Poraba EUR/leto	Visoka poraba
Mojca	1890	32	Ž	18200	da
Janez	1200	48	M	8900	ne
Špela	900	63	Ž	9200	da

Opredelitev cilja

Visoka poraba pomeni vsaj 9000 EUR nakupov v naslednjih 12 mesecih.
Napovedujemo lahko znesek porabe ali kategorijo da/ne.

Ime je identifikator. Prihodnje porabe ne uporabimo kot vhod za napoved kategorije, saj ob napovedi še ni znana.

Napovedni model

Model je funkcija

$$\mathcal{X} = D_{X_1} \times D_{X_2} \times \cdots \times D_{X_p}, \qquad m : \mathcal{X} \to D_Y, \qquad \hat{y} = m(\boldsymbol{x}).$$

Iz vrednosti napovednih spremenljivk izračuna napoved cilja.

Dve napovedni nalogi

- Regresija: napoved numerične vrednosti, npr. letne porabe.
- Klasifikacija: napoved kategorije, npr. visoka poraba: da/ne.

Naloga nadzorovanega učenja

Prostor podatkov vsebuje končne sezname

$$\mathcal{D} = \bigcup_{n \geq 1} (\mathcal{X} \times D_Y)^n.$$

Povprečno izgubo računamo le na nepraznih seznamih.

Algoritem in model

$\mathcal{M} \subseteq D_Y^{\mathcal{X}}$: družina kandidatnih modelov. Učni algoritem $\mathcal{A} : \mathcal{D} \to \mathcal{M}$ iz podatkov sestavi model:

$$\mathcal{A} \colon S_{\text{train}} \mapsto m, \qquad m \colon x \mapsto \hat{y}.$$

Iščemo model, ki dobro napoveduje na novih opazovanjih.

Izguba in napovedna napaka

Izguba ene napovedi

$$L : D_Y \times D_Y \to \mathbb{R}_{\geq 0}, \qquad L(y, \hat{y}).$$

Pri spodnjih izgubah je $L(y, \hat{y}) = 0$ natanko tedaj, ko $y = \hat{y}$.

Običajni izbiri

$$L_{SE}(y, \hat{y}) = (y - \hat{y})^2, \qquad L_{01}(y, \hat{y}) = \mathbb{1}(y \neq \hat{y}).$$

Kvadratna izguba za regresijo; izguba 0–1 za klasifikacijo. Alternativa pri regresiji je absolutna izguba $|y - \hat{y}|$, ki je manj občutljiva na osamelce in v ničli ni odvedljiva.

Napovedna napaka na podatkovni množici S

$$\text{Error}(m, S) = \frac{1}{n} \sum_{i=1}^{n} L(y_i, m(\boldsymbol{x}_i)), \qquad n > 0.$$

Povprečna izguba na seznamu primerov

Vsak pojav primera prispeva en člen vsote, tudi če se vrednosti ponovijo.

Zakaj je razlika med seznamom in množico pomembna?

Za cilje $(2, 2, 5)$ in konstantno napoved $\hat{y} = 2$ je MSE enaka $(0 + 0 + 9)/3 = 3$.

Če eno ponovitev odstranimo, dobimo drugačne podatke in drugačno napako: $(0 + 9)/2 = 4.5$.

Prileganje in posploševanje

Prileganje

Majhna Error(m, S_{train}) pomeni dobro napovedovanje na učnih podatkih.

Posploševanje

Želimo majhno napako na novih opazovanjih iz okolja uporabe. Ocenimo jo na ločenih podatkih S_{test}.

Novo opazovanje ima lahko enake vrednosti kot učni primer. Majhna učna napaka sama še ne zagotavlja dobre testne napake.

Učna, validacijska in testna množica

- S_{train}: učenje modela in prilagoditev priprave podatkov.
- S_{val}: izbira algoritma in njegovih hiperparametrov.
- S_{test}: končna ocena izbranega algoritma.

Ločitev po indeksih opazovanj

Za $I_{\text{train}}, I_{\text{val}}, I_{\text{test}} \subseteq \{1, 2, \ldots, n\}$ zahtevamo paroma disjunktne množice indeksov. $S_{\text{train}} = (e_i)_{i \in I_{\text{train}}}$ in podobno za drugi delitvi.

Enake vrednosti se lahko pojavijo v različnih delitvah. Povezane zapise delimo po skupinah; pri časovnih podatkih upoštevamo čas.

Algoritem strojnega učenja $\mathcal{A}$

Nadzorovano učenje

$m = \mathcal{A}(S_{\text{train}}; \lambda)$: iz vhodov in ciljev sestavimo model. $\lambda \in \Lambda_{\mathcal{A}}$ označuje nastavitve hiperparametrov, npr. število sosedov k.

Parametri in hiperparametri

Parametre modela, npr. koeficiente β linearne regresije, se naučimo iz podatkov. Hiperparametre izbiramo z validacijo.

Nenadzorovano učenje

Algoritem iz vhodov odkrije strukturo, npr. skupine podobnih primerov. Podanih ciljnih vrednosti ni.

Ocenjevanje algoritma in nastavitev

Ocena s prečnim preverjanjem na S

$$\mathrm{CV}(\mathcal{A}, \lambda, S) = \frac{1}{K} \sum_{k=1}^{K} \mathrm{Error}(\mathcal{A}(S_{-k}; \lambda), S_k).$$

S_k: validacijski seznam v k-ti delitvi; S_{-k}: preostali podatki. Delitve imajo enako velikost; sicer povprečje utežimo po številu primerov.

V celotnem predavanju: manjša napaka je boljša

Ocene na podatkih za učenje so praviloma optimistične. Pri prečnem preverjanju tudi pripravo podatkov prilagodimo le na S_{-k}.

Končno oceno dobimo z ločenim testom ali gnezdenim prečnim preverjanjem.

Algoritmi za nadzorovano strojno učenje

- Linearni modeli: linearna in logistična regresija.
- Metoda najbližjih sosedov.
- Odločitvena drevesa in pravila.
- Ansambli: vrečenje, pospeševanje in naključni gozdovi.
- Metode podpornih vektorjev in jedrne funkcije.
- Umetne nevronske mreže.

Priporočena priprava za nadaljevanje

Ponovitev osnov

Gradivo predmeta Analiza podatkov s strojnim učenjem (prej *Izbrane teme iz analize podatkov*): podatki, modeli, izgube in vrednotenje.

Dodatno branje

Prosto dostopen učbenik: *An Introduction to Statistical Learning*

Pravila igre

Pregled vsebin

Štirje tematski sklopi naprednega strojnega učenja

1. Uvod in meta učenje
2. Globoko učenje (5 tednov, vabljeno predavanje)
3. Grafi in strukturirani podatki (2 tedna)
4. Strojno učenje in diskretna optimizacija (3 tedni, vabljeno predavanje)

Način izvajanja predmeta

Tematski sklop

- Predavanja in seminarji, 3 ure na teden
- Vaje, 2 uri na teden
- Domača naloga

Vsebina predavanj in seminarjev

- Predavanja: predstavitev sklopa s poudarkom na algoritmih
- Seminarji: raziskave, odprta vprašanja in vabljena predavanja

Ocenjevanje: domače naloge (DN)

Domače naloge, največ 50 odstotnih točk

- Ena domača naloga na koncu posameznega tematskega sklopa
- Vsako DN izvajamo v dveh delih, doma in v računalniški učilnici
- Prvi del: implementacije in naloge, ki jih oddate v spletni učilnici
- Drugi del: naloge, ki jih rešujete v učilnici brez UI
- Pravočasna oddaja prvega dela je pogoj za reševanje drugega dela
- Implementacije iz prvega dela uporabljate pri reševanju drugega dela
- Drugi del prevladuje pri ocenjevanju domače naloge

- Uspešno opravljene naloge: doseženih več kot 25 točk
- Podrobnosti in roke za oddajo bomo objavljali v spletni učilnici

Ocenjevanje: ustni izpit (UI)

Ustni izpit, največ 50 odstotnih točk

- Vprašanja in naloge iz vseh tematskih sklopov
- Vprašanja so lahko teoretična ali praktična

- Pogoj za pristop so uspešno opravljene domače naloge
- Uspešno opravljen ustni izpit: več kot 25 točk

Končna ocena, največ 100 odstotnih točk: seštevek točk iz DN in UI

Meta učenje

Vsebina

- Meta učenje je strojno učenje o strojnem učenju
- V nadaljevanju današnjega predavanja

Globoko učenje

Vsebina

- Arhitekture in učenje sodobnih nevronskih mrež

- Rekurenčne nevronske mreže in mehanizmi pozornosti
- Samokodirniki, transformerji in generativni modeli
- Veliki jezikovni modeli

Grafi in strukturirani podatki

Vsebina

- Podatki iz relacijskih podatkovnih baz in grafov

- Od besedil do grafov: vložitve vozlišč grafa node2vec
- Vložitve povezav in grafov, grafovske nevronske mreže
- Rekurzivne nevronske mreže

Strojno učenje in diskretna optimizacija

Vsebina

- Uporaba strojnega učenja pri optimizaciji

- Simbolna regresija, odkrivanje enačb iz podatkov
- Simbolna regresija z globokim učenjem
- Bayesovska optimizacija, AutoML
- Spodbujevalno učenje

Izvajalci in literatura

Izvajalci

- Predavanja in seminarji: Ljupčo Todorovski
- Vaje: Sebastian Mežnar
- Domače naloge: Sebastian Mežnar
- Ustni izpit: Ljupčo Todorovski

Literatura

- Za vsak sklop posebej: učbeniki in članki
- Povezave in seznami v spletni učilnici

Neobstoj brezplačnega kosila (NBK)

Ali je en algoritem vedno najboljši?

Izrek o neobstoju brezplačnega kosila

Če vse možne Boolove ciljne funkcije obravnavamo kot enako verjetne, imajo vsi učni algoritmi enako pričakovano izgubo 0–1 na testnih podatkih.

Kaj moramo pri trditvi določiti?

Prostor ciljnih funkcij, njihovo porazdelitev in način merjenja napake. Dokazali bomo preprosto različico za končen prostor brez šuma.

Vir: Wolpert: The Supervised Learning No-Free-Lunch Theorems.

Predpostavke izreka

Končen prostor vhodov in vseh ciljnih funkcij

$$B = \{0,1\}, \qquad \mathcal{X} = B^p, \qquad \mathcal{F} = B^{\mathcal{X}}, \qquad |\mathcal{F}| = 2^{2^p}.$$

Ciljno funkcijo f naključno izberemo iz $\mathcal{F}$, enakomerna porazdelitev.

Učni vhodi so določeni neodvisno od f

Fiksiramo vhode $(\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_n)$. Ciljna funkcija f določi učni seznam:

$$S_f = ((\boldsymbol{x}_1, f(\boldsymbol{x}_1)), (\boldsymbol{x}_2, f(\boldsymbol{x}_2)), \ldots, (\boldsymbol{x}_n, f(\boldsymbol{x}_n))).$$

$U = \{\boldsymbol{x}_i : i = 1, 2, \ldots, n\}$ je množica različnih učnih vhodov. Zahtevamo $T = \mathcal{X} \setminus U \neq \emptyset$.

Izrek: pričakovana napaka na testnih vhodih

Za poljuben deterministični algoritem $\mathcal{A}$ naj bo $m_f = \mathcal{A}(S_f)$. Definiramo napako na testnih vhodih:

$$R_T(\mathcal{A}, f) = \frac{1}{|T|} \sum_{\boldsymbol{x} \in T} \mathbb{1}(m_f(\boldsymbol{x}) \neq f(\boldsymbol{x})).$$

Trditev za poljubna algoritma $\mathcal{A}$ in $\mathcal{B}$

$$\frac{1}{|\mathcal{F}|} \sum_{f \in \mathcal{F}} R_T(\mathcal{A}, f) = \frac{1}{|\mathcal{F}|} \sum_{f \in \mathcal{F}} R_T(\mathcal{B}, f) = \frac{1}{2}.$$

Merimo samo napake zunaj U; napake na učnih vhodih se lahko razlikujejo.

Dokaz: pari ciljnih funkcij

Izberemo poljuben testni vhod $\boldsymbol{x} \in T$. Za vsako $f \in \mathcal{F}$ definiramo

$$f_{\boldsymbol{x}}(\boldsymbol{z}) = \begin{cases} 1 - f(\boldsymbol{x}), & \boldsymbol{z} = \boldsymbol{x}, \\ f(\boldsymbol{z}), & \boldsymbol{z} \neq \boldsymbol{x}. \end{cases}$$

Isti učni podatki, nasprotna cilja na $\boldsymbol{x}$

Ker $\boldsymbol{x} \notin U$, velja $S_f = S_{f_{\boldsymbol{x}}}$. Zato je $m_f = m_{f_{\boldsymbol{x}}}$: isti model in ista napoved na $\boldsymbol{x}$.

Ena od obeh napovedi je napačna, torej velja:

$$\mathbb{1}(m_f(\boldsymbol{x}) \neq f(\boldsymbol{x})) + \mathbb{1}(m_f(\boldsymbol{x}) \neq f_{\boldsymbol{x}}(\boldsymbol{x})) = 1.$$

Dokaz: povprečenje po ciljnih funkcijah

Za fiksen $\boldsymbol{x} \in T$ označimo izgubo

$$\ell_f(\boldsymbol{x}) = \mathbb{1}\big(m_f(\boldsymbol{x}) \neq f(\boldsymbol{x})\big).$$

Ker je $m_f = m_{f_{\boldsymbol{x}}}$, zadnja enačba prejšnje prosojnice pravi $\ell_f(\boldsymbol{x}) + \ell_{f_{\boldsymbol{x}}}(\boldsymbol{x}) = 1$. Povprečimo jo po vseh $f \in \mathcal{F}$:

$$\frac{1}{|\mathcal{F}|}\sum_{f\in\mathcal{F}} \ell_f(\boldsymbol{x}) + \frac{1}{|\mathcal{F}|}\sum_{f\in\mathcal{F}} \ell_{f_{\boldsymbol{x}}}(\boldsymbol{x}) = 1.$$

Preslikava $f \mapsto f_{\boldsymbol{x}}$ je bijekcija, saj $(f_{\boldsymbol{x}})_{\boldsymbol{x}} = f$. Obe vsoti zato vsebujeta iste izgube, le v drugem vrstnem redu:

$$\frac{2}{|\mathcal{F}|}\sum_{f\in\mathcal{F}} \ell_f(\boldsymbol{x}) = 1 \quad\Longrightarrow\quad \frac{1}{|\mathcal{F}|}\sum_{f\in\mathcal{F}} \ell_f(\boldsymbol{x}) = \frac{1}{2}.$$

Dokaz: povprečenje po testnih vhodih

Za vsak $\boldsymbol{x} \in T$ je povprečna izguba po vseh funkcijah $1/2$. Zdaj izračunamo povprečno napako na celotni množici T:

$$\begin{aligned}
\frac{1}{|\mathcal{F}|}\sum_{f\in\mathcal{F}} R_T(\mathcal{A}, f) &= \frac{1}{|\mathcal{F}|}\sum_{f\in\mathcal{F}}\left[\frac{1}{|T|}\sum_{\boldsymbol{x}\in T}\ell_f(\boldsymbol{x})\right] && \text{(definicija } R_T\text{)}\\
&= \frac{1}{|T|}\sum_{\boldsymbol{x}\in T}\left[\frac{1}{|\mathcal{F}|}\sum_{f\in\mathcal{F}}\ell_f(\boldsymbol{x})\right] && \text{(zamenjava vsot)}\\
&= \frac{1}{|T|}\sum_{\boldsymbol{x}\in T}\frac{1}{2} = \frac{1}{|T|}\,|T|\,\frac{1}{2} = \frac{1}{2}.
\end{aligned}$$

Vsoti sta končni; T je ista množica za vse f, ker so učni vhodi fiksni.

Dokaz uporablja le enakost napovedi pri istih učnih podatkih. Za naključni algoritem fiksiramo njegovo naključnost in nato povprečimo tudi po njej.

Ponazoritev: isti učni podatki, različna cilja

	X_1	X_2	f	$f_{(1,0)}$	napoved modela
učni vhod	0	0	1	1	1
učni vhod	0	1	0	0	0
testni vhod	1	0	0	1	1
testni vhod	1	1	1	1	0

Na izbranem vhodu $(1, 0)$

Ista napoved 1 ima pri f izgubo 1, pri $f_{(1,0)}$ pa izgubo 0. Če bi model napovedal 0, bi se izgubi zamenjali.

Prikazani model je le primer. Za poljuben algoritem sta učna seznama ista, zato sta tudi naučena modela enaka.

Kaj izrek pomeni za prakso?

Primerjava je odvisna od problemov

Algoritem je lahko boljši na določenem razredu nalog. Izrek izenači algoritme pri enakomernem povprečenju po vseh ciljnih funkcijah, ne na posamezni nalogi.

Predpostavke omogočajo posploševanje

Realni problemi imajo strukturo, npr. gladkost ali lokalno podobnost. Induktivna pristranskost algoritma določa, katere pravilnosti izkorišča.

Izrek ne določa, kateri algoritem je najboljši za naše podatke. Izbiro utemeljimo s poznavanjem problema in ustreznim vrednotenjem.

Od NBK do meta učenja

Učenje iz izkušenj z več nalogami

Pri realnih nalogah ciljnih funkcij ne izbiramo enakomerno. Pretekle naloge pokažejo, kateri algoritmi so uspešni pri podobnih problemih.

Osnovno vprašanje meta učenja

Ali lahko iz lastnosti nove podatkovne množice napovemo, kateri učni algoritem bo primeren zanjo?

Prenos izkušenj je koristen, če so si pretekle in nove naloge dovolj podobne.

Meta učenje

Meta učenje: strojno učenje o strojnem učenju

Dve ravni primerov

Pri običajnem učenju je primer opazovanje, npr. kupec. Pri tukaj obravnavanem meta učenju je primer učna naloga, predstavljena s podatkovno množico S.

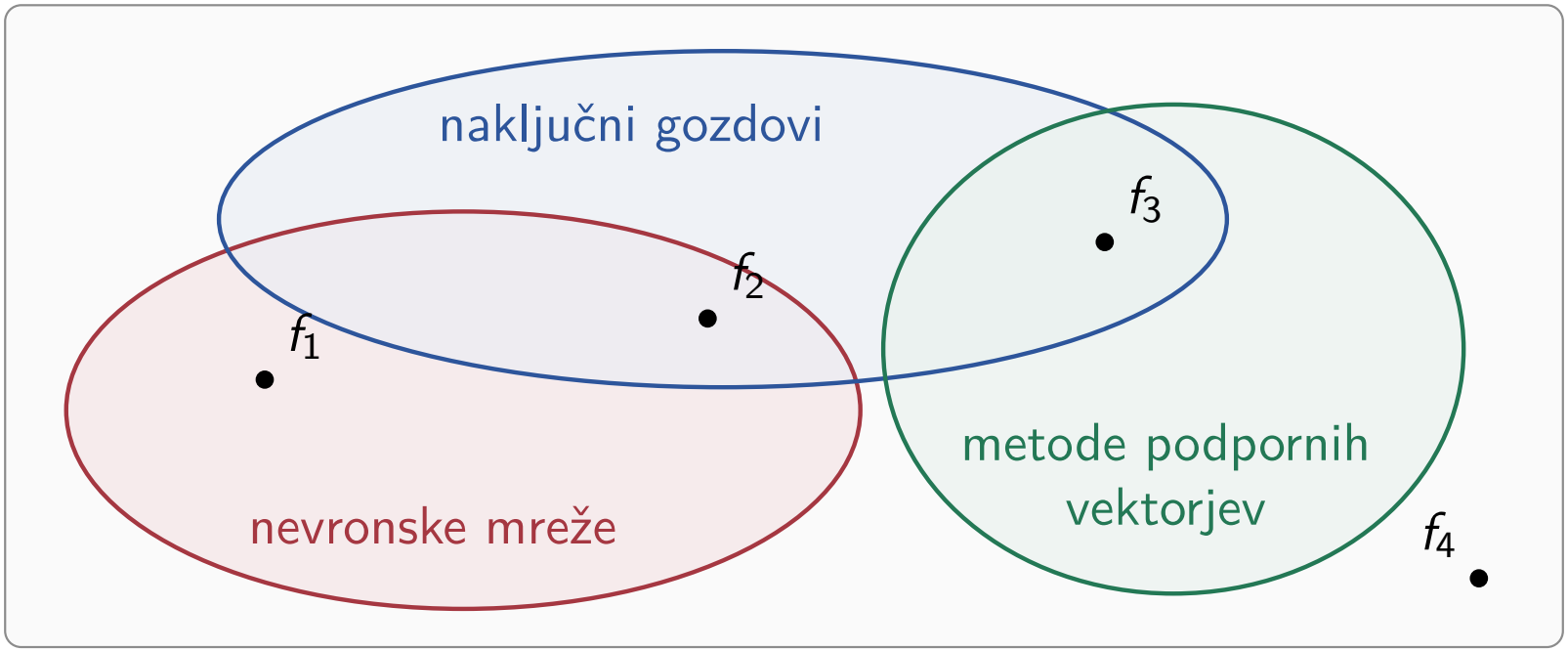
Dve vrsti meta spremenljivk

- Meta napovedne spremenljivke: lastnosti podatkovne množice.
- Meta ciljne spremenljivke: uspešnost ali izbira učnega algoritma.

Meta model poveže lastnosti naloge z izkušnjami o uspešnosti algoritmov.

Področja ekspertize v prostoru funkcij

Različne družine algoritmov izkoriščajo različne pravilnosti ciljnih funkcij.

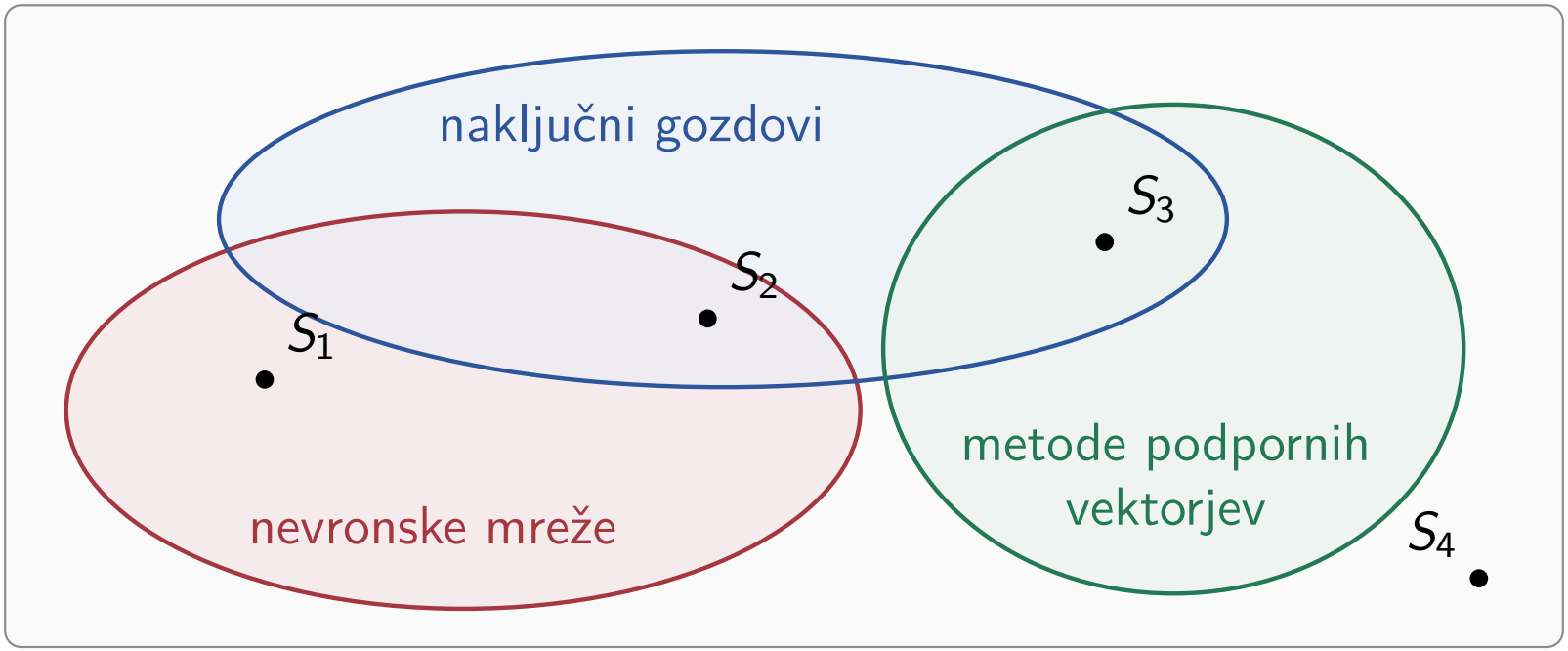


Točke $f_i \in \mathcal{F}$ so ciljne funkcije; obarvana področja ponazarjajo, kje je posamezna družina algoritmov lahko uspešna.

Shematski prikaz: področja se prekrivajo; njihove meje so odvisne od podatkov, nastavitev algoritmov in izbrane mere napake.

Področja ekspertize v prostoru podatkov

Opazujemo podatkovno množico S; ciljna funkcija f je neznana.



Točke $S_i \in \mathcal{D}_{\text{meta}}$ so podatkovne množice. Meta spremenljivke nam pomagajo prepoznati naloge s podobnimi lastnostmi.

Shematski prikaz: področja ekspertize ocenimo iz preteklih poskusov. Na uspešnost vplivajo tudi velikost podatkovne množice, šum in tipi spremenljivk.

Meta napovedne spremenljivke

Od podatkovne množice do vektorskega opisa

$$\text{meta}_X : \mathcal{D}_{\text{meta}} \to \mathbb{R}^{\tilde{p}}, \qquad S \mapsto \tilde{\boldsymbol{x}} = (\tilde{x}_1, \tilde{x}_2, \ldots, \tilde{x}_{\tilde{p}}).$$

$\tilde{X}_j$: meta napovedna spremenljivka; $\tilde{x}_j$: njena vrednost za podano nalogo.
$\tilde{p}$: število meta napovednih spremenljivk.

Naloge imajo lahko različne dimenzije

$\mathcal{D}_{\text{meta}}$ vsebuje podatkovne množice za izbrani razred nalog, ki se lahko razlikujejo po številu primerov n, spremenljivk p in domenah. Njihovi meta opisi imajo skupno dolžino $\tilde{p}$.

Osnovne meta napovedne spremenljivke

Velikost in sestava podatkovne množice

- Število primerov n in napovednih spremenljivk p; razmerje n/p.
- Število numeričnih, kategoričnih in dvojiških spremenljivk.
- Delež manjkajočih vrednosti v podatkovni tabeli.

Ciljna spremenljivka pri klasifikaciji

Število opazovanih razredov in delež večinskega razreda:

$$\hat{\pi}_v = \frac{1}{n}\sum_{i=1}^{n} \mathbb{1}(y_i = v), \qquad \max_{v \in D_Y} \hat{\pi}_v.$$

Vsaka ponovitev primera prispeva svoj člen v vsoti.

Preprost opis velikosti, sestave in neuravnoteženosti podatkov.

Statistične meta napovedne spremenljivke

Porazdelitve numeričnih vrednosti

- Povprečje, mediana in standardni odklon.
- Kvartili, asimetrija in sploščenost porazdelitve.
- Povzetki cilja Y in korelacije z njim pri regresiji.

Povezave med spremenljivkami

Primer agregiranja: povprečna absolutna korelacija med numeričnimi vhodi:

$$\frac{2}{p_n(p_n-1)} \sum_{1 \le j < k \le p_n} |\hat{\rho}_{jk}|.$$

$p_n > 1$: število numeričnih vhodov; $\hat{\rho}_{jk}$: vzorčna korelacija.

Numerične vhode indeksiramo od 1 do p_n. Za manjkajoče vrednosti in konstantne stolpce določimo pravilo izračuna.

Informacijske meta napovedne spremenljivke

Entropija kategorične spremenljivke

Za $\hat{\pi}_v$, delež primerov z vrednostjo $Y = v$, izračunamo

$$\hat{H}(Y) = -\sum_{v \in D_Y} \hat{\pi}_v \log_2 \hat{\pi}_v, \qquad 0 \log_2 0 := 0.$$

Entropijo izračunamo tudi za kategorične napovedne spremenljivke X_j.

Vzajemna informacija z razredom

$$\widehat{\mathrm{MI}}(X_j, Y) = \hat{H}(X_j) + \hat{H}(Y) - \hat{H}(X_j, Y).$$

Opiše povezanost kategoričnega vhoda s ciljem. Meta opis lahko vsebuje povprečje ali maksimum teh vrednosti po stolpcih.

Verjetnosti ocenimo s frekvencami v seznamu S, vključno s ponovitvami.

Meta napovedne spremenljivke: mere kompleksnosti

Geometrija in ločljivost podatkov

- Razdalje med centri razredov in razpršenost znotraj razredov.
- Prekrivanje razredov v prostoru napovednih spremenljivk.
- Delež primerov, katerih najbližji sosed ima drug razred.

Notranja dimenzija podatkov

Število glavnih komponent, ki pojasnijo vsaj 95 % variance, je primer opisa efektivne dimenzije numeričnih podatkov. Uporabimo lahko tudi ocene notranje ali fraktalne dimenzije.

Mere so odvisne od kodiranja in merila spremenljivk. Zahtevnejši izračuni so smiselni le, če pomagajo pri izbiri algoritma.

Modelske meta napovedne spremenljivke

Podatkovno množico opišemo prek naučenega modela

Za dogovorjen algoritem in nastavitve zgradimo $m = \mathcal{A}(S; \lambda)$ ter izmerimo lastnosti njegove strukture.

Strukturne lastnosti modelov

- Odločitveno drevo: globina, število vozlišč in listov.
- Drevo: povprečna globina listov in velikost pripadajočih skupin.
- Metoda podpornih vektorjev: število podpornih vektorjev.
- Redek linearni model: delež neničelnih koeficientov.

Dogovor o učenju, npr. rezanju drevesa ali regularizaciji, mora biti enak pri vseh nalogah, ki jih primerjamo.

Algoritmične meta napovedne spremenljivke

Orientirji (*landmarks*)

Nalogo opišemo z validacijsko napako hitrih učnih algoritmov tudi na manjših vzorcih. Za algoritem $\mathcal{A}_j$ in nastavitve λ_j izračunamo:

$$\tilde{x}_j = \mathrm{CV}(\mathcal{A}_j, \lambda_j, S).$$

Primeri: odločitveni štor, en najbližji sosed, linearni model.

Relativni in vzorčni orientirji

- Relativni: razlika napak dveh izbranih algoritmov.
- Vzorčni: napaka algoritma na manjšem vzorcu primerov.
- Več velikosti vzorca: kratek opis učne krivulje.

Napake ocenimo na ločenih primerih; učna napaka bi bila optimistična.
Strošek orientirjev mora biti manjši od koristi pri usmerjanju iskanja.

Kaj pričakujemo od meta opisa?

- Primerljive vrednosti na različnih podatkovnih množicah.
- Lastnosti, uporabne za napoved meta cilja.
- Nizko računsko ceno glede na prihranek pri izbiri algoritma.
- Jasno obravnavo različnih tipov in manjkajočih vrednosti.

Na novi nalogi uporabimo le razpoložljive podatke

Meta opis izračunamo iz podatkov, ki so na voljo ob izbiri algoritma. Za izbiro ne uporabimo končnega testnega dela nove naloge.

Meta ciljna spremenljivka

Cilj izhaja iz rezultatov preteklih poskusov

$$\text{meta}_Y : \mathcal{D}_{\text{meta}} \to D_{\tilde{Y}}, \qquad S \mapsto \tilde{y}.$$

$\tilde{Y}$: meta ciljna spremenljivka; $\tilde{y}$: njena vrednost za nalogo S. Preslikava predpostavlja dogovorjene kandidate in postopek vrednotenja.

Primer: napovedovanje napake določenega algoritma

Za fiksna $\mathcal{A}$ in λ določimo

$$\tilde{y} = \text{CV}(\mathcal{A}, \lambda, S).$$

To je regresijska naloga na meta ravni. Meta model oceni napako tega algoritma na novi nalogi brez celotnega preizkusa.

Iz preteklih nalog sestavimo meta podatke

Seznam nalog in seznam meta primerov

$$\mathbb{S}_{\text{train}} = (S_1, S_2 \ldots, S_N), \qquad \tilde{S} = (\tilde{e}_1, \tilde{e}_2, \ldots, \tilde{e}_N),$$

$$\tilde{e}_i = (\tilde{\boldsymbol{x}}_i, \tilde{y}_i) = (\text{meta}_X(S_i), \text{meta}_Y(S_i)).$$

N: število meta primerov; i: indeks naloge; j: indeks meta stolpca.

Tabela ima N vrstic in $\tilde{p}$ napovednih stolpcev

Ena vrstica opisuje eno nalogo. Tudi meta podatki so seznam: enaki opisi ali primeri se lahko ponovijo.

Podatki o uspešnosti učnih algoritmov

	$(\mathcal{A}_1, \lambda_1)$	$(\mathcal{A}_2, \lambda_2)$	$\cdots$	$(\mathcal{A}_r, \lambda_r)$
S_1	c_{11}	c_{12}	$\cdots$	c_{1r}
S_2	c_{21}	c_{22}	$\cdots$	c_{2r}
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
S_N	c_{N1}	c_{N2}	$\cdots$	c_{Nr}

$$c_{ij} = \mathrm{CV}(\mathcal{A}_j, \lambda_j, S_i).$$

Primerljivost poskusov

Na isti nalogi uporabimo enake delitve, izgubo in pravila priprave podatkov.
Pri različnih nalogah upoštevamo tudi razlike v merilu ciljnih vrednosti.

Poskuse izvedemo sami ali uporabimo primerljive rezultate iz OpenML.
Manjkajoč rezultat ni ničelna napaka.

Učenje in uporaba meta modela

Učenje iz preteklih nalog

$$\mathbb{S}_{\text{train}} \xrightarrow{\text{meta}_X,\text{meta}_Y} \tilde{S} \xrightarrow{\tilde{\mathcal{A}}} \tilde{m}.$$

Izberemo meta algoritem $\tilde{\mathcal{A}}$, primeren tipu cilja $\tilde{Y}$.

Napoved za novo nalogo

$$S_{\text{new}} \xrightarrow{\text{meta}_X} \tilde{\boldsymbol{x}}_{\text{new}} \xrightarrow{\tilde{m}} \hat{\tilde{y}}.$$

Meta model $\tilde{m}$ preslika meta opis v napoved meta cilja. Na novi nalogi nam za to napoved ni treba izračunati meta_Y.

Izbira in razvrščanje učnih algoritmov

Izbira najboljšega kandidata

Za r kandidatov s fiksnimi nastavitvami določimo

$$j^*(S) = \underset{j \in \{1,\ldots,r\}}{\arg\min}\ \mathrm{CV}(\mathcal{A}_j, \lambda_j, S).$$

Meta cilj $\tilde{y} = j^*(S)$ je kategorija: klasifikacijska naloga. Pri izenačenju uporabimo dogovorjeno pravilo.

Priporočilo več kandidatov

Napovemo lahko tudi napake vseh kandidatov in jih uredimo od najmanjše do največje. Tako dobimo razvrstitev algoritmov, ki jih je smiselno preizkusiti najprej.

Izbira hiperparametrov

Meta cilj: dobra nastavitev za podano nalogo

$$\lambda^*(S) = \arg\min_{\lambda \in \Lambda_{\mathcal{A}}} \mathrm{CV}(\mathcal{A}, \lambda, S).$$

V praksi iščemo najboljšo med preizkušenimi nastavitvami, ob omejenem času za poskuse.

Kaj lahko prenesemo s preteklih nalog?

Meta model priporoči začetne nastavitve ali obetavno območje iskanja. Njihovo kakovost nato preverimo na novi nalogi.

Povezava med meta učenjem in AutoML

AutoML avtomatizira izbiro učnega algoritma

Išče pripravo podatkov, algoritem in hiperparametre, ki pri omejenem računskem proračunu dosežejo majhno validacijsko napako.

Meta učenje usmeri iskanje z izkušnjami

pretekle naloge $\longrightarrow$ meta model $\longrightarrow$ začetni kandidati za novo nalogo.

AutoML priporočene kandidate ovrednoti na novi nalogi in nadaljuje iskanje, npr. z Bayesovsko optimizacijo.

Meta učenje lahko pospeši AutoML; AutoML lahko deluje tudi brez njega.

Vir: Vanschoren: Meta-Learning, v knjigi AutoML (2019).

Kako preverimo meta model?

Ločimo naloge za učenje in vrednotenje

Meta model ocenimo na nalogah, ki niso sodelovale pri njegovem učenju.
Različic iste izvorne podatkovne množice ne razdelimo med obe strani.

Merimo korist priporočila

Primerjamo doseženo napako in računski čas z izbiro brez meta učenja.
Dobra napoved meta cilja mora pomagati tudi pri reševanju nove naloge.

Končni testni podatki nove naloge ostanejo ločeni od izbire algoritma.

Sorodne ideje: prenos in večopravilno učenje

Prenosno učenje (*transfer learning*)

Model ali predstavitev, naučena na eni nalogi, je izhodišče za učenje na drugi, povezani nalogi.

Večopravilno učenje (*multi-task learning*)

Več povezanih nalog se učimo skupaj, npr. s skupno predstavitvijo in ločenimi napovednimi izhodi.

Širše meta učenje lahko uči tudi predstavitve ali začetne parametre za hitro prilagajanje; danes smo obravnavali predvsem izbiro algoritmov.

Literatura in praktični napotki

Priporočena literatura

- Wolpert (2002): NBK za nadzorovano učenje.
- Vanschoren (2019): okvir meta učenja in povezava z AutoML.
- Rivolli in sod. (2018): pregled meta napovednih spremenljivk.

Programska oprema in podatki

- PyMFE: izračun meta napovednih spremenljivk.
- OpenML: podatkovne množice in rezultati poskusov.